THE UNIVERSITY OF WARWICK

Paper Details

Paper Code: CS2600

Paper Title: Algorithms

Exam Period: April 2025

Exam Rubric

Time allowed: 2 hours

Exam Type: Standard Examination

Instructions

Answer **ONE** question from Section A and **TWO** questions from Section B.

Indicate clearly which three questions you have answered. Among the questions attempted, only the first one in Section A and the first two in Section B (in their order of appearance in the answer book) will be marked.

Read carefully the instructions on the answer booklet and make sure that the particulars required are entered on each answer booklet.

Calculators are not allowed.
The exam question paper MUST NOT be removed from the examination venue.

WARWICK
THE UNIVERSITY OF WARWICK

Section A Answer **ONE** question from this section.

1. This question revisits several problems that were studied in lectures.

   (a) In the stable matching problem, doctors (A, B, C, ...) rank hospitals (Z, Y, X, ...), and vice-versa. We seek a perfect matching of doctors to hospitals such that there are no *unstable pairs*. An unstable pair (d, h) is one where doctor d prefers h to their assigned hospital h', and hospital h prefers d to their assigned doctor d'. Give an example of a perfect matching that is not stable among two doctors A and B and two hospitals Z and Y, and justify your answer. [6]

   (b) The furthest-in-the-future scheduling technique aims to minimize the number of cache misses for the offline caching problem. Given an ordered sequence of cache requests, it will eject from the cache the item that has the longest time until its next request. Show a sequence of requests that maximizes the number of cache misses under the furthest-in-the-future policy for a cache of size $k = 2$. [6]

   (c) Consider a connected graph G with edge costs that are all distinct. Alicia thinks that the edge with *lowest* cost must be included in every minimum spanning tree of G. Alfie thinks that the edge with *highest* cost must never be included in any minimum spanning tree of G. Which of these statements is correct, and why? [6]

   (d) The SUBSET-SUM Problem (SSP) is, given a set S of n integers and a target value T, to determine whether there exists a subset of S whose sum is exactly T. SSP is well-known to be NP-complete. SSP can be solved by a dynamic programming approach in time $O(nT)$. Explain why this does not imply P = NP. [6]

   (e) The Bellman-Ford algorithm can be used to find the length of all shortest paths in a weighted, directed graph G ending at a chosen vertex t, when the graph can have edges with negative weights, but there are no cycles of negative weight. The algorithm performs $n - 1$ steps of dynamic programming, where n is the number of vertices in the graph. Explain how observing the steps of the Bellman-Ford algorithm we can detect if there are any cycles of negative weight in the graph. [6]

Total for Question 1: 30

2. This question is on the relationships between complexity classes.

Let P be the set of decision problems solvable in polynomial time, and NP to be as defined in lectures.

(a) Prove that P $\subseteq$ NP. [6]

Let PSPACE be the set of decision problems that can be solved using at most polynomial amounts of space (and unbounded time).

(b) Prove that P $\subseteq$ PSPACE. [6]

(c) Prove that NP $\subseteq$ PSPACE. [6]

* * *

A problem is considered to be NP-*equivalent* if it is not a decision problem, but it is both "NP-easy" (it can be solved by polynomial reduction to an NP-complete problem) and "NP-hard" (any problem in NP can be solved by reduction to it).

The SUBSET-SUM Problem (SSP) is, given a set S of n integers and a target value T, to determine whether there exists a subset of S whose sum is exactly T. SSP is well-known to be NP-complete.

Let FIND-SUBSET-SUM be the analogous search problem: return either a specific subset of S whose sum is exactly T, or "No" if none exists.

(d) Prove that the FIND-SUBSET-SUM search problem is NP-equivalent. [12]

Total for Question 2: 30

Section B Answer **TWO** questions from this section.

3. This question is on the tractability of algorithmic problems.

   (a) Describe what is meant by the operator $\leq_P$ in terms of relationships between algorithmic problems. [5]

   (b) Prove rigorously that the relation described by the $\leq_P$ operator is transitive. [5]

   Consider a universe U and a set S of subsets of U. A set cover $S' \subseteq S$ is a subset of S such that the union of S' is U (it "covers" the universe).

   The SET-COVER decision problem asks, given a universe U and a set of subsets S, whether there is a set cover S' of size at most k. SET-COVER is well known to be complete for NP.

   Recall that a problem is considered to be *tractable* if it admits a polynomial-time algorithm.

   (c) Is SET-COVER tractable if k is fixed in advance (i.e. k is not part of the input)? [5]

   (d) Let $F = (U, E)$ be a forest—an undirected graph with no cycles. Is SET-COVER tractable when $S = E$? [10]

   (e) Is SET-COVER tractable if we enforce that no two sets in S have the same cardinality? [10]

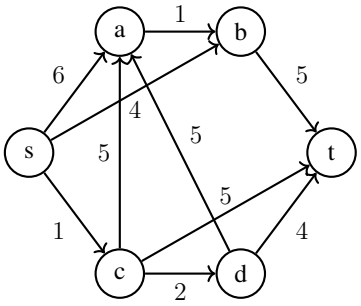
Total for Question 3: 35

4. This question is on flow networks.

   A flow network $N = (V, E, s, t, c)$ is a finite directed graph with distinguished *source* and *sink* vertices, and edges weighted by positive integers according to the capacity function c.

   Let the *fragility* of a flow network N be the minimum number of edges we can remove from N such that the max flow is reduced to 0.

   (a) Consider the following flow network. What is its fragility? [7]



   (b) Give an algorithm to compute the fragility of a given flow network, and state and justify its running time. [7]

   (c) Describe all flow networks $N = (V, E, s, t, c)$ such that the fragility of N equals $|E|$. [7]

* * *

Consider two flow networks $A = (V, E_A, s, t, c_A)$ and $B = (V, E_B, s, t, c_B)$ over the same set of vertices V and with the same source and sink vertices. We can *superimpose* A and B to form $A \mathbin{@} B$ by adding together the edges from both flow networks. Formally:

$$A \mathbin{@} B = (V, E, s, t, c) \qquad E = E_A \cup E_B \qquad c(e) = \begin{cases} c(a) + c(b) & e \in E_A \cap E_B \\ c(a) & e \in E_A \setminus E_B \\ c(b) & e \in E_B \setminus E_A \end{cases}$$

   (d) Clara says that the max flow in $A \mathbin{@} B$ is always *at most* the sum of the max flows in A and B. Is Clara right? [7]

   (e) Donna says that the max flow in $A \mathbin{@} B$ is always *at least* the sum of the max flows in A and B. Is Donna right? [7]

Total for Question 4: 35

5. This question is about divide-and-conquer algorithms.

   The Stoogesort algorithm sorts a list of n items as follows:

   - If the first item is larger than the last item, then swap them.
   - If the list has three or more elements then:
     - Stoogesort the first 2/3 of the list
     - Stoogesort the final 2/3 of the list
     - Stoogesort the first 2/3 of the list again

   The algorithm rounds up fractional values, i.e., it takes $\lceil 2n/3 \rceil$ items in each recursive call.

   (a) Apply the Stoogesort algorithm to the input $[1, 3, 2]$ and show the steps that it takes. How many item comparisons does it make? [4]

   (b) Write a recurrence relation for the number of item comparisons made by the Stoogesort algorithm, and solve this to give an expression of the form $\Theta(n^{\log_y x} \log^z n)$ for the number of comparisons as a function of n. [8]

   (c) The Bubblesort algorithm makes $O(n^2)$ item comparisons. Should the Stoogesort be used in preference to the Bubblesort algorithm? [4]

   The Walsh-Hadamard transform is an operation performed on a vector via matrix multiplication. For a column vector $\mathbf{v}$ of dimension 2^n, the Walsh-Hadamard transform of $\mathbf{v}$ is the vector $H_n\mathbf{v}$. The matrix H_0 is $\begin{bmatrix}1\end{bmatrix}$ and H_{n+1} is defined recursively as $\begin{bmatrix} H_n & H_n \\ H_n & -H_n \end{bmatrix}$.

   (d) Compute the Walsh-Hadamard transform of the vector $\mathbf{w} = \begin{bmatrix}2 & 0 & 2 & 5\end{bmatrix}^\mathsf{T}$. [4]

   (e) One way to perform the Walsh-Hadamard transform for a vector $\mathbf{v}$ is as follows: [7]

   1. Apply the above definition recursively to build the matrix H_n
   2. Compute the matrix-vector multiplication $H_n\mathbf{v}$

   What is the running time of this algorithm?

   (f) An alternative method to compute the Walsh-Hadmard transform of vector $\mathbf{v}$ of dimension 2^n is to split it into two halves, $\mathbf{v}_0$ of the first 2^{n-1} entries, and $\mathbf{v}_1$ of the last 2^{n-1} entries. The Walsh-Hadamard transform is then given by $\begin{bmatrix}(H_{n-1}\mathbf{v}_0) + (H_{n-1}\mathbf{v}_1) \\ (H_{n-1}\mathbf{v}_0) - (H_{n-1}\mathbf{v}_1)\end{bmatrix}$. [8]

   What is the running time of an algorithm that recursively applies this splitting method, as a function of n?

Total for Question 5: 35

End